



DriveX Pakistan
LUXURY RENTALS

— PREMIUM MOBILITY PLATFORM —

DriveX Pakistan

A frontend-only luxury car rental platform engineered with production-grade behaviour, dynamic pricing intelligence, and a fully simulated business backend.

DOCUMENT

[README](#)

STACK

[Next.js](#) · [TypeScript](#) · [Tailwind v4](#)

ARCHITECTURE

[Frontend-only](#) · [Backend-grade](#)

YEAR

[2026](#)

DOCUMENT ARCHITECTURE TEAM

LEAD DEVELOPER

[Muhammad Shams Ul Arfeen](#)

CO-DEVELOPER

[Sabeeka Kainat Zahra](#)



00

Table of Contents

A guided walkthrough of the DriveX project — from architectural decisions and the design system, through customer experience, business intelligence, and engineering challenges, to the future roadmap.

01	Project Overview & Vision	03
02	Tech Stack & Why We Chose It	04
03	System Architecture	05
04	Design System & Visual Language	06
05	Customer Experience & Features	07
06	Business Logic & Pricing Intelligence	08
07	Data Management & Persistence	09
08	Engineering Challenges & Solutions	10
09	Roadmap & Future Enhancements	11
010	Conclusion & Credits	12

One-line summary — DriveX simulates a complete, real-world car rental SaaS using only the browser. There is no server, no database, no hosting cost — yet every interaction (auth, bookings, dynamic pricing, admin operations, cross-tab sync) feels production-grade.



01

Project Overview

DriveX Pakistan is a luxury car rental web application designed to replicate a real-world premium mobility business operating across six major Pakistani cities. The platform delivers an end-to-end customer journey — discovery, search, booking, payment, and post-purchase tracking — alongside a fully featured admin control panel.

— The problem we set out to solve

Most rental platforms in Pakistan are template-driven, visually inconsistent, and operationally rigid. Pricing is static, promo codes are exposed, payment flows feel unfinished, and admins have no real control over fleet, fees, or discounts. DriveX was built as the antithesis — an editorial-grade interface paired with a flexible, dynamic, admin-controlled engine.

— What makes DriveX different

- **Frontend-only, backend-grade.** Zero server cost, yet every store behaves like a real database with reactive subscriptions and cross-tab synchronization.
- **Dynamic pricing intelligence.** Service fees and weekend / holiday uplifts are layered transparently — users always see the base price, never a hidden surcharge.
- **Hidden promo system.** Codes work silently in the background; the UI never advertises them, so only users with prior knowledge can redeem one — exactly like real luxury platforms.
- **Self-healing Order IDs.** Globally unique numeric IDs that survive counter resets, tampering, or device migration.
- **Editorial design.** A black-gold-white palette with Cormorant Garamond + Inter typography, glassmorphism, and gold-shimmer accents.

— Who it is for

This document is written for evaluators, future maintainers, and reviewers who want to understand the entire system at a glance — without reading source code. Every section maps directly to a real implementation decision in the codebase.



02

Tech Stack & Why

A deliberately minimal toolkit. Two primary languages, one framework, and a small set of focused libraries — each chosen for a specific, defensible reason.

TOOL	ROLE	WHY WE PICKED IT
TypeScript	Language	Strict types catch entire classes of bugs at compile time. Every store, engine, and component is typed end-to-end.
Next.js 16	Framework	App Router gives us file-based routing, server components, and Turbopack out of the box.
React 19	UI library	Stable <code>useSyncExternalStore</code> made our localStorage-as-database pattern reactive and safe.
Tailwind CSS v4	Styling	Utility-first design system; theme tokens defined once in <code>globals.css</code> , used everywhere.
shadcn/ui	UI primitives	Production-quality components (Button, Dialog, Input, Tabs) that live inside the project — fully customisable.
Lucide	Icon set	Clean, minimal stroke icons that align perfectly with the editorial aesthetic.
Sonner	Notifications	Slim, accessible toast system for booking confirmations and validation feedback.
Recharts	Visualisation	Powers the admin analytics — revenue trends, booking volume, fleet utilisation.
localStorage	Persistence	Browser-native key-value store, treated as a JSON document database via custom store wrappers.

The two main languages — TypeScript writes every line of logic and UI; Tailwind v4 writes every line of styling. That is the entire technical surface. Everything else is a focused



03

System Architecture

DriveX is structured as three clean layers. Each layer has one responsibility, talks to the next through a typed interface, and never reaches across boundaries.

1 Presentation Layer

React server & client components in `app/` and `components/`. Pages compose feature components; feature components compose UI primitives.



2 Logic & Engine Layer

Pure TypeScript modules in `lib/` — discount engine, fees engine, booking engine. Stateless calculators that take inputs and return decisions.



3 Persistence Layer

Reactive localStorage stores with snapshot caching, cross-tab `storage` events, and React subscriptions via `useSyncExternalStore`.

— Folder organisation

<code>app/</code>	Routes — home, cars, checkout, profile, admin, login, register
<code>components/</code>	Customer-side UI & admin panel sections
<code>lib/</code>	Stores, engines, types — the entire backend simulation
<code>hooks/</code>	Reusable React hooks (mobile detection, toast)
<code>public/</code>	Static assets, presentation deck, printable docs

— Why this structure scales

Adding a new feature (say, a loyalty tier) requires only three predictable touch points: a new store in `lib/`, an engine modification, and a UI component. No backend deployment, no schema migration, no API versioning — the entire system updates atomically.



04

Design System

Every visual decision flows from one editorial brief — a luxury black, gold, and white palette inspired by automotive print catalogues. The system is consistent, accessible, and tokenised.

PALETTE

Black · Gold · White

Three colours form the entire surface. Gold (`#d4af37`) is reserved for accents and brand moments; never overused.

TYPOGRAPHY

Cormorant + Inter

Cormorant Garamond delivers editorial elegance in headlines; Inter handles every functional detail in the UI.

EFFECTS

Glass + Shimmer

Glassmorphism cards with subtle borders, gold-shimmer text, and gold-glow shadows reinforce the premium feel.

— Motion language

Animations are restrained but deliberate — fade + slide entrances, hover lifts on cards, scale-down on press, and staggered reveals on long lists. Nothing bounces; nothing distracts. Every transition uses a soft cubic-bezier curve to feel organic.

— Component philosophy

- **Accessibility first.** All interactive elements have keyboard support, ARIA roles, and focus states.
- **Mobile-first responsiveness.** Layouts collapse gracefully on narrow viewports without losing hierarchy.

- **Single source of truth.** Every colour, radius, font, and shadow is a CSS custom property — change it in `globals.css` and the entire app updates.

— Premium signal moments

The cover, hero section, success screen, and admin dashboard each include one carefully placed "wow" moment — animated success ring with confetti burst, gold-shimmer order numbers, and a slowly drifting gradient background — designed to feel deliberate, never decorative.



05

Customer Experience

The customer journey is a deliberately short, premium funnel — from landing page to confirmed booking in under a minute, with every checkpoint reinforcing trust.

01

Discover

Hero search with city + date pickers, six city tiles, featured fleet carousel, and editorial banner.

02

Browse

Filterable fleet (82 cars across 6 categories), front/back hover swap on every car card, real-time availability.

03

Inspect

Detail page with full image gallery, spec sheet, pricing, and a "book this car" CTA that pre-fills the checkout.

04

Authenticate

Register / login with name, email, phone, CNIC. Sessions persist via localStorage; logout clears state cleanly.

05

Checkout

06

Confirm

Live price breakdown, promo input, 4 payment methods, per-method validation, staged processing pipeline.

Animated success screen with confetti, gold-shimmer order ID, full receipt, and a path to the profile dashboard.

— Trust signals throughout

- Base car price is **always** visible — fees and discounts appear as separate line items, never merged.
- Promo errors return one neutral message ("Invalid or expired") to prevent code enumeration.
- Form validation is immediate and field-specific; users never see false errors.
- The success screen never lies — bookings are saved *before* the celebration animation runs.



06

Business Logic

Behind the editorial surface sits a precise calculation pipeline. Every booking flows through the same deterministic stages, with admin-controlled levers at every step.

— Pricing pipeline (worked example)

Base price (PKR 10,000 × 2 days)	PKR 20,000
Loyalty discount (5% returning customer)	– PKR 1,000
Service fee (5%, applied after discount)	+ PKR 950
Weekend uplift (5%, Saturday pickup)	+ PKR 950
Total	PKR 20,900

— Admin control surfaces

- **Fleet management** — add, edit, deactivate cars; per-city distribution; category tagging.

- **Discount layers** — loyalty thresholds, bulk multipliers, category-wide rules, seasonal windows.
- **Promo codes** — full CRUD, expiry, usage limits, category restrictions, minimum-order rules.
- **Service fees** — toggle on/off, percentage, weekend/holiday uplift with named special dates.
- **Booking workflow** — pending → confirmed → cancelled transitions with audit trail.

— Self-healing Order IDs

Order numbers start at 100001 and increment monotonically. The allocator floors every new ID at `max(counter+1, highestExisting+1, 100001)` — so even if the counter is wiped, tampered with, or restored from a stale backup, the next ID is guaranteed to be strictly greater than every booking already in storage.



07

Data Management

localStorage is treated as a JSON document database. Each store is a typed wrapper that exposes read, write, subscribe, and reactive React hook APIs.

— Storage schema

KEY	SHAPE	OWNER
<code>drivex_users_v1</code>	User[]	auth-store
<code>drivex_session_v1</code>	{ userId, token }	auth-store
<code>drivex_cars_v1</code>	Car[]	cars-store
<code>drivex_cities_v1</code>	City[]	city-store
<code>drivex_bookings_v1</code>	Booking[]	bookings-store
<code>drivex_order_counter_v1</code>	number	bookings-store
<code>drivex_discount_settings_v1</code>	DiscountSettings	discount-settings

— Reactive store pattern

Every store exposes `read()`, `write()`, `subscribe()`, and a `useStore()` React hook built on `useSyncExternalStore`. Snapshot caching ensures stable object references — without this, React would see "new" state on every render and trigger an infinite re-render loop. Cross-tab synchronisation is automatic via the browser's native `storage` event.

— Integrity guarantees

- Append-only bookings — past records are never overwritten.
- Versioned keys (`_v1` suffix) allow safe future migrations.
- Defensive parsing — corrupt JSON falls back to typed defaults instead of crashing.



08

Engineering Challenges

Real bugs encountered during development — and the precise fixes that resolved them. Each problem is an opportunity to demonstrate the team's debugging discipline.

1 Maximum update depth exceeded

CAUSE `useSyncExternalStore` snapshot getters returned a freshly parsed object on every render, triggering an infinite re-render loop on the checkout page.

FIX Module-level snapshot cache keyed by the raw JSON string. Re-parse only on actual change; pre-warm the cache on every write so the next read returns the same identity.

2 Checkout payment regression

CAUSE No try/catch around `addBooking`; quota errors in private-mode browsers froze the spinner. Wallet methods had no input UI, so empty data slipped through.

FIX Staged pipeline (validating → authorizing → saving → confirmed) wrapped in try/catch/finally. Per-method validation. Inputs disabled during submission. JazzCash and EasyPaisa each got proper phone + PIN flows.

3 Duplicate Order IDs

CAUSE Counter resets and storage tampering could produce duplicate IDs across users.

FIX Self-healing allocator floors every new ID at `max(counter+1, highestExistingOrderNumber+1, 100001)`.

4 Wrong car images

CAUSE A Suzuki Cultus VXR entry used guessed image URLs that did not exist in the verified dataset.

FIX Strict per-car image policy enforced. The unverified entry was removed; the legitimate Cultus VXL remained intact. Misassigned imagery was repurposed as a homepage decorative banner.

5 Promo code over-exposure

CAUSE Suggested codes shown in the UI removed the exclusivity associated with promotional offers.

FIX Hidden by design. The input shows only "Enter Promo Code". All failure paths collapse into a single neutral "Invalid or expired" message — preventing enumeration attacks and preserving exclusivity.



09

Roadmap

DriveX is intentionally complete as a frontend simulation. The path to a live, multi-tenant SaaS product is clear and incremental — every layer below can be replaced without rewriting the UI.

— Near-term (next iteration)

- **Real backend integration.** Swap the persistence layer for Supabase or Neon. The store interfaces remain identical.
- **Live payment gateways.** Replace simulated card / wallet flows with Stripe, JazzCash Business API, and EasyPaisa Merchant API.
- **Email + SMS notifications.** Booking confirmations, reminders, and admin alerts via Resend and a local SMS provider.

— Mid-term

- **Native mobile app.** React Native build sharing the same engines and types.
- **AI-assisted concierge.** Natural-language booking ("a luxury sedan in Lahore for the weekend") backed by a tool-using LLM agent.
- **Driver-on-demand.** Optional chauffeur booking with real-time matching.
- **Multi-currency & multi-language.** Urdu / Roman Urdu / English locale switching.

— Long-term vision

Open the platform to fleet partners — independent rental businesses can list their cars, manage availability, and receive bookings through a partner dashboard. DriveX becomes the rails, not just the brand.



10

Conclusion

DriveX Pakistan demonstrates that a serious, production-feeling SaaS product can be built entirely on the front-end without compromising on rigour, polish, or maintainability.

— What was achieved

- ✓ An editorial, accessible, mobile-first interface across 8 distinct route groups.
- ✓ A reactive store pattern that turns localStorage into a real-time multi-tab database.
- ✓ A dynamic, admin-controlled pricing engine with transparent line-item breakdown.
- ✓ A self-healing Order ID allocator and a hardened, staged payment pipeline.
- ✓ A complete admin operations suite — fleet, fees, discounts, promos, bookings.

— What was learned

Production-grade engineering is less about which framework you pick and more about the discipline of small, predictable abstractions. Three layers, typed end-to-end, with one source of truth per concern — and the system stays understandable as it grows.

Final thought — DriveX is not a tutorial project. It is an opinionated, finished product simulation. Every decision was made deliberately, every bug was investigated to its root cause, and every visual flourish reinforces the brand's premium positioning.

— Credits

Built with Next.js 16, React 19, TypeScript, Tailwind CSS v4, shadcn/ui, Lucide, Sonner, and Recharts. Designed and engineered with attention to detail, in 2026.

— Team

Muhammad Shams Ul Arfeen & **Sabeeka Kainat Zahra**